

Core Java Training

Day 7

Topics

Abstraction

- abstract classes
- interfaces

Abstraction in Java

Abstraction is the process of **hiding internal implementation** and showing only **essential features** to the user.

It allows to focus on **"what an object does"** rather than **"how it does it"**. The unnecessary details are not displayed to the user.

Real-Life Analogy - TV Remote Control:

- You know what buttons to press (volumeUp(), powerOn()).
- You don't know how the circuit works internally.

That's abstraction — only important things are exposed, unnecessary details are hidden.

Why Use Abstraction?

- To define a **common structure** or contract
- To **reduce complexity** by hiding low-level logic
- To **decouple implementation** from usage

Abstraction in Java

Java provides two ways to implement abstraction, which are listed below:

- **Abstract Classes (Partial Abstraction)**
- **Interface (100% Abstraction)**

1. **Abstract Classes:** An abstract class is a class that you can't instantiate and can only extend by subclasses. Abstract classes can have both abstract and non-abstract methods. Abstract methods do not have a body and you must implement them by any subclass that extends the abstract class. Non-abstract methods have a body and you can directly call them by the subclass.
2. **Interfaces:** An interface is a collection of methods. You can use it to define a set of behaviors that a class should implement. A class can implement multiple interfaces, and all the methods defined in an interface must be implemented by any class that implements it.

Abstraction in Java

The abstract keyword is a non-access modifier, used for classes and methods:

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

An abstract class can have both abstract and regular methods:

```
abstract class Animal {
  public abstract void animalSound();
  public void sleep() {
    System.out.println("Zzz");
  }
}
```

From the example above, it is not possible to create an object of the Animal class:

Real-World Problem : Online Payment Gateway

You are designing an **online shopping app**. Users can pay via **Credit Card, UPI, or Wallet**. You want to define a **common structure**, but let each payment type **handle its own logic**.

Step 1: Abstract Class - Payment

```
abstract class Payment {
    double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    abstract void makePayment();   // abstract method

    void generateReceipt() {
        System.out.println("Receipt generated for ₹" +
amount);
    }
}
```

Real-World Java Problem : Online Payment Gateway

Step 2: Subclasses - CreditCardPayment, UPIPayment, etc.

```
class CreditCardPayment extends Payment {
    CreditCardPayment(double amount) {
        super(amount);
    }
    @Override
    void makePayment() {
        System.out.println("Paying ₹" + amount
+ " using Credit Card...");
    }
}
```

```
class UPIPayment extends Payment {
    UPIPayment(double amount) {
        super(amount);
    }
    @Override
    void makePayment() {
        System.out.println("Paying ₹" +
amount + " using UPI...");
    }
}
```

Real-World Java Problem : Online Payment Gateway

Step 3: Driver Code - Main Class

```
public class Main {
    public static void main(String[] args) {
        Payment p1 = new CreditCardPayment(500.00);
        p1.makePayment();
        p1.generateReceipt();

        Payment p2 = new UPIPayment(300.00);
        p2.makePayment();
        p2.generateReceipt();
    }
}
```

Question 2: E-commerce Order Delivery

Different delivery modes (**Standard** and **Express**) have different charges.

```
// Abstract class
abstract class Delivery {
    abstract void deliveryCharge(double amount);

    public void orderPlaced() {
        System.out.println("Order placed
successfully!");
    }
}

// Standard delivery
class StandardDelivery extends Delivery {
    @Override
    void deliveryCharge(double amount) {
        System.out.println("Standard Delivery Charge:
₹" + (amount * 0.05));
    }
}
```

```
// Express delivery
class ExpressDelivery extends Delivery {
    @Override
    void deliveryCharge(double amount) {
        System.out.println("Express Delivery Charge: ₹" +
(amount * 0.10));
    }
}
public class Main {
    public static void main(String[] args) {
        Delivery d1 = new StandardDelivery();
        Delivery d2 = new ExpressDelivery();

        d1.orderPlaced();
        d1.deliveryCharge(1000);

        d2.orderPlaced();
        d2.deliveryCharge(1000);
    }
}
```

Question 3: Hospital Management

A hospital has different types of doctors: **Dentist** and **Cardiologist**.

Each doctor has a different specialization but must implement a **treatPatient()** method.

```
// Abstract Class
abstract class Doctor {
    String name;

    Doctor(String name) {
        this.name = name;
    }
    abstract void treatPatient();

    public void hospitalInfo() {
        System.out.println("Hospital: City Care
Hospital");
    }
}
// Dentist
class Dentist extends Doctor {
    Dentist(String name) {
        super(name);
    }
    void treatPatient() {
        System.out.println(name + " is treating a dental issue.");
    }}
// Cardiologist
class Cardiologist extends Doctor {
    Cardiologist(String name) {
        super(name);}
    @Override
    void treatPatient() {
        System.out.println(name + " is treating a heart issue.");
    }}
public class Main {
    public static void main(String[] args) {
        Doctor d1 = new Dentist("Dr. Sharma");
        Doctor d2 = new Cardiologist("Dr. Mehta");
        d1.hospitalInfo();
        d1.treatPatient();
        d2.hospitalInfo();
        d2.treatPatient();
    }}
```

Problem Statement:

Design and implement a Java-based **Library Management System** using **Object-Oriented Programming concepts**. **Functional Requirements:**

- Add books to library
- Register new member (Student/Faculty)
- Issue book to member (check max limit)
- Return book
- List all books
- List all issued books

Tips:

- Use Interfaces like Loanable, Displayable
- Create Exception handling for invalid book issue
- Track number of books issued per member
- Use toString() to format object output

Practice Questions:

Q1. Which of the following is true about abstract classes in Java?
A, You can instantiate an abstract class directly
B. Abstract classes can have constructors.
C. Abstract methods must have a body
D. Abstract class can't have implemented methods

Q2. What happens if a class contains an abstract method but is not declared abstract?
A. It runs successfully
B. Compiler error
C. Runtime error
D. The method is ignored

Q3. Can you declare an abstract method as static?
A. Yes
B. No
C. Only from Java 9
D. Only inside interfaces

Practice Questions:

Q4. Which of these modifiers **cannot** be used with an abstract method?
A. public
B. protected
C. private
D. abstract

Q5. Which of these is a valid way to use an abstract class?
A. Instantiate it directly
B. Extend it and override abstract methods
C. Use it to create constants
D. Implement it like an interface

Q6. Can an abstract class implement an interface without providing method implementations?
A. Yes
B. No
C. Only if the class is final
D. Only in Java 8+

Practice Questions:

Q7. What will happen if a subclass of an abstract class doesn't implement all abstract methods?

A. Compilation error
B. Program runs with warnings
C. It becomes an abstract class itself
D. JVM adds method definitions automatically

Q8. Which statement is true for an abstract class in Java?

A. It must have at least one abstract method
B. It cannot have any constructors
C. It can have both abstract and non-abstract methods
D. It must implement all interfaces

Q9. Abstract classes are primarily used for:

A. Preventing inheritance
B. Hiding class variables
C. Providing a base for subclasses with default behavior
D. Improving performance

Practice Questions:

What will be the output?

A. Animal speaks

B. Dog barks

C. Animal speaks

Dog barks

D. Compilation error

```
class Animal {
    void speak() {
        System.out.println("Animal speaks");
    }
}

class Dog extends Animal {
    void speak() {
        super.speak();
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.speak();
    }
}
```

Practice Questions:

What will be printed?

A. Shape constructor called

B. Circle constructor called

C. Shape constructor called

Circle constructor called

D. Nothing

```
abstract class Shape {
    Shape() {
        System.out.println("Shape constructor
called");
    }
}
class Circle extends Shape {
    Circle() {
        System.out.println("Circle constructor
called");
    }
}
public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
    }
}
```

Practice Questions:

Which OOP concept is used here?

A. Overriding

B. Polymorphism (Compile-time)

C. Abstraction

D. Encapsulation

```
class Print {
   void show(int a) {
      System.out.println("Integer: " + a);
   }

   void show(String s) {
      System.out.println("String: " + s);
   }
}

public class Test {
   public static void main(String[] args) {
      Print p = new Print();
      p.show("Java");
      p.show(10);
   }
}
```

Practice Questions:

What will be the output?

A. Hello from A

Hello from B

Hello from MyClass

B. Hello from MyClass

C. Compilation error

D. Hello from A

```
interface A {
    default void greet() {
        System.out.println("Hello from A");
    }
}
interface B {
    default void greet() {
        System.out.println("Hello from B");
    }
}
class MyClass implements A, B {
    public void greet() {
        A.super.greet();
        B.super.greet();
        System.out.println("Hello from MyClass");
    }
}
public class Test {
    public static void main(String[] args) {
        new MyClass().greet();
    }
}
```

Practice Questions:

What does this code demonstrate?

A. Inheritance

B. Polymorphism

C. Encapsulation

D. Abstraction

```
class Person {
    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

public class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.setName("Ravi");
        System.out.println(p.getName());
    }
}
```

Practice Questions:

What is the purpose of *this* in constructor?

A. Refers to the base class

B. B. Refers to current class object

C. C. Refers to local variable

D. D. None of the above

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " +
name);
    }
}
```

Key tips abstract classes

1. An instance of an abstract class can not be created.
2. Constructors are allowed.
3. We can have an abstract class without any abstract method.
4. There can be a **final method** in abstract class but any abstract method in class(abstract class) can not be declared as final or in simpler terms final method can not be abstract itself as it will yield an error: "Illegal combination of modifiers: abstract and final"
5. We can define static methods in an abstract class
6. We can use the **abstract keyword** for declaring ***top-level classes (Outer class) as well as inner classes*** as abstract
7. If a **class** contains at least **one abstract method** then compulsory should declare a class as abstract
8. If the **Child class** is unable to provide implementation to all abstract methods of the **Parent class** then we should declare that **Child class as abstract** so that the next level Child class should provide implementation to the remaining abstract method

Interface

An **interface** is a **blueprint of a class**.

It contains only **abstract methods** (methods without a body) and **constants** (implicitly `public static final`).

A class must **implement** an interface to provide method definitions.

Supports **multiple inheritance** (a class can implement multiple interfaces).

```
interface Animal {
    void sound(); // abstract method (public + abstract by default)
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
```

- Methods in an interface are **public abstract** by default.
- Variables in an interface are **public static final** (constants).
- A class **uses** **implements keyword** to use an interface.
- A class can **implement multiple interfaces**.
- From **Java 8 onwards**, interfaces can have:
- **default methods** (with body)
- **static methods**

Interface

A completely abstract type used to specify a contract — what a class must do, but not how.

To access the interface methods, the interface must be "implemented" (kinda like inherited) by another class with the implements keyword (instead of extends).

```
interface Animal {
  public void animalSound(); // interface
method (does not have a body)
  public void sleep(); // interface method
(does not have a body)
}

// Pig "implements" the Animal interface
class Pig implements Animal {
  public void animalSound() {
    // The body of animalSound() is
provided here
    System.out.println("The pig says: wee
wee");
  }
```

```
 public void sleep() {
    // The body of sleep() is provided
here
    System.out.println("Zzz");
  }
}

class Main {
  public static void main(String[]
args) {
    Pig myPig = new Pig();  // Create a
Pig object
    myPig.animalSound();
    myPig.sleep();
  } }
```

Key Features of Java

Multiple Interfaces: To implement multiple interfaces, separate them with a comma:

```
interface FirstInterface {
  public void myMethod(); // interface method
}

interface SecondInterface {
  public void myOtherMethod(); // interface method
}

class DemoClass implements FirstInterface, SecondInterface
{
  public void myMethod() {
    System.out.println("Some text..");
  }
public void myOtherMethod() {
    System.out.println("Some other text...");
  }
}

class Main {
  public static void main(String[] args) {
    DemoClass myObj = new DemoClass();
    myObj.myMethod();
    myObj.myOtherMethod();
  }
}
```

Question 1: Multiple Interfaces

Problem: Create two interfaces `Printable` and `Showable`. A class `Report` should implement both and display messages.

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}
class Report implements Printable, Showable {
    public void print() {
        System.out.println("Printing Report...");
    }
    public void show() {
        System.out.println("Showing Report...");
    }
}
public class Main {
    public static void main(String[] args) {
        Report r = new Report();
        r.print();
        r.show();
    }
}
```

Question 2: Basic Interface Implementation
Problem: Create an interface Shape with a method area(). Implement it in Circle and Rectangle classes.

```
interface Shape {
    void area();
}
class Circle implements Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    public void area() {
        System.out.println("Circle area: " + (Math.PI * radius
* radius));
    }
}
class Rectangle implements Shape {
    private double length, width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    public void area() {
        System.out.println("Rectangle area: " +
(length * width));
    }
}

public class Main {
    public static void main(String[] args) {
        Shape c = new Circle(5);
        Shape r = new Rectangle(4, 6);
        c.area();  // Circle area: 78.54
        r.area();  // Rectangle area: 24.0
    }
}
```

Question 3: Interface for Banking Transactions

Problem: Create an interface BankOperations with deposit() and withdraw(). Implement it in BankAccount class.

```
interface BankOperations {
    void deposit(double amount);
    void withdraw(double amount);
}

class BankAccount implements BankOperations {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    public void withdraw(double amount) {
        if(amount <= balance) {
            balance -= amount;
            System.out.println("Withdrew: " + amount);
        } else {
            System.out.println("Insufficient funds!");
        }
    }
    public void showBalance() {
        System.out.println("Balance: " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        acc.deposit(500);
        acc.withdraw(200);
        acc.showBalance();
    }
}
```

Question 4: Interface with Default Method (Java 8+)
Problem: Create an interface Vehicle with an abstract method start() and a default method stop(). Implement it in Car.

```
interface Vehicle {
    void start();

    default void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
        c.stop();
    }
}
```

Difference b/w Abstract and Interface

Feature	Abstract Class	Interface
Keyword Used	abstract class	interface
Method Types	Both **abstract** and **concrete** methods	Only abstract (Java 7), from Java 8: **default s static** methods
Fields (Variables)	Can have **instance variables** with access modifiers	All variables are **public static final** (constants)
Constructors	✅ Yes	❌ No constructors
Multiple Inheritance	❌ Not allowed (Java doesn't support it with classes)	✅ A class can implement **multiple interfaces**
Access Modifiers Allowed	public, protected, private	Only public (everything is implicitly public abstract)
Performance	Slightly faster	Slightly slower due to indirection (especially older versions)
When to Use	When classes share common behavior/fields	When unrelated classes must follow same **contract**
Java Version Enhancements	Minor additions	Java 8+: default, static methodsJava 9+: private methods